Relatório Técnico: Implementação de Pipeline de Dados em Tempo Real (DataOps)

Projeto: Iniciação Científica - DataOps

Data: 17 de Dezembro de 2025

Autor: Gabriel Gonçalves Sampaio - Engenharia de Computação (UNIFEI)

1. Visão Geral do Pipeline

O objetivo desta etapa foi estabelecer um fluxo de ingestão de dados contínuo (Streaming) para monitoramento climático. A arquitetura implementada segue o padrão:

1. **Fonte:** API NASA POWER (Dados meteorológicos e solares).
2. **Ingestão/Producer:** Script Python simulando sensores IoT em múltiplas cidades (SP, Itajubá, RJ, BH).
3. **Message Broker:** Apache Kafka (Gerenciamento de filas e buffer).
4. **Armazenamento OLAP:** Apache Druid (Análise em tempo real).

---

2. Configuração do Ambiente e Rede (Docker)

Para viabilizar a comunicação entre os serviços containerizados, foi necessário configurar uma rede Docker compartilhada (druid-net) e ajustar os *listeners* do Kafka para aceitar conexões internas (do Druid) e externas (do Script Python no host).

- **Rede:** druid-net (Bridge).
- **Kafka Bootstrap Internal:** kafka:29092 (Usado pelo Druid).
- **Kafka Bootstrap External:** localhost:9092 (Usado pelo Python).

---

3. Etapas de Integração

3.1. Estruturação e Envio dos Dados (Producer)

Foi desenvolvido um script em Python que coleta dados históricos e em tempo real, serializando-os em formato JSON.

- **Tópico Criado:** clima-sudeste-raw
- **Estrutura do Payload:** JSON contendo dimensões (city, sensor_id) e métricas (temperature_c, solar_irradiance).

Ponto Chave: O formato de data original da NASA (YYYYMMDDHH como inteiro) exigiu tratamento especial, sendo enviado no campo timestamp.

3.2. Configuração de Ingestão no Apache Druid ("Load Data")

A conexão foi realizada via console web do Druid, utilizando o supervisor de indexação para Kafka.

Passo A: Conexão com o Bootstrap Server

O Druid foi configurado para "escutar" o Kafka através da rede interna do Docker.

- *Configuração:* kafka:29092
- *Tópico:* clima-sudeste-raw

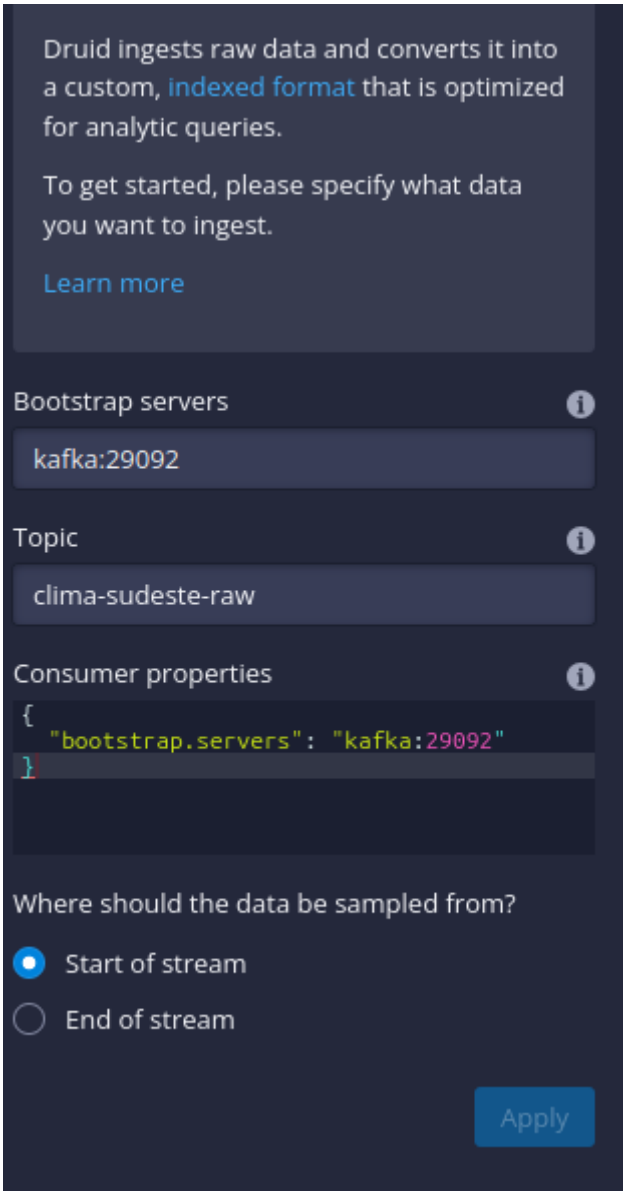


Figura 1 - Preview dos dados brutos chegando do Kafka no console do Druid.

Passo B: Parseamento de Dados e Timestamp

Um dos desafios técnicos foi o reconhecimento da coluna de tempo. O Druid não detectou automaticamente o formato yyyyMMddHH, exigindo uma spec customizada.

- *Coluna de Tempo:* timestamp
- *Formato Aplicado:* Custom -> yyyyMMddHH

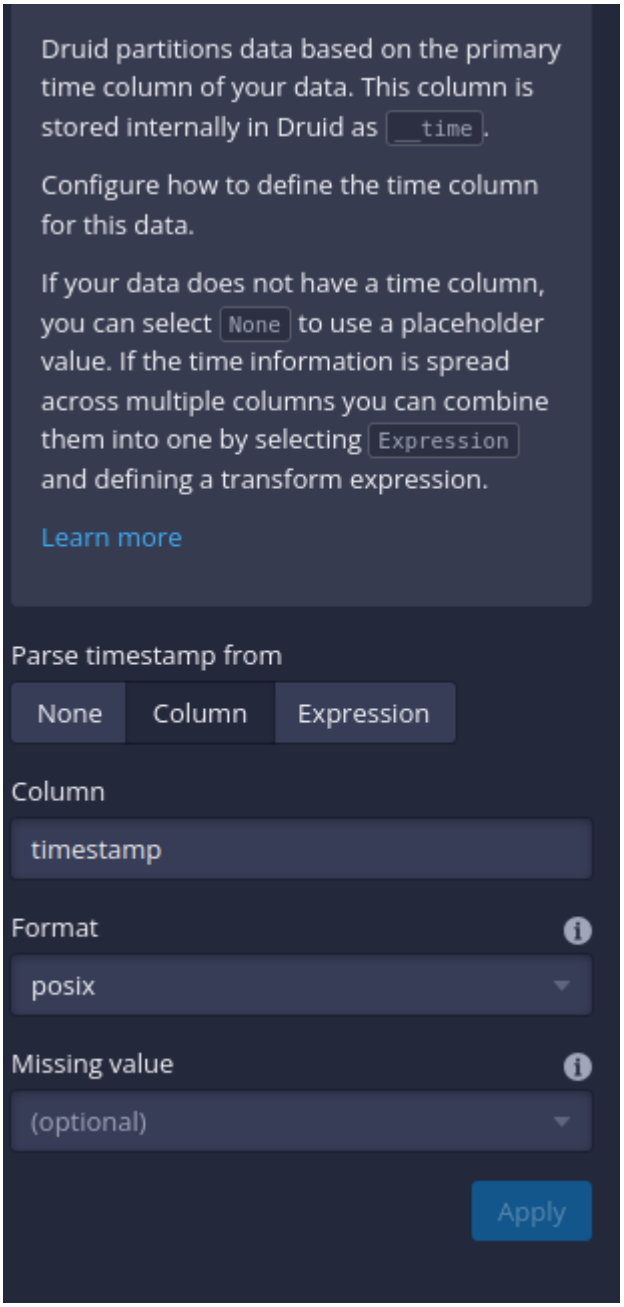
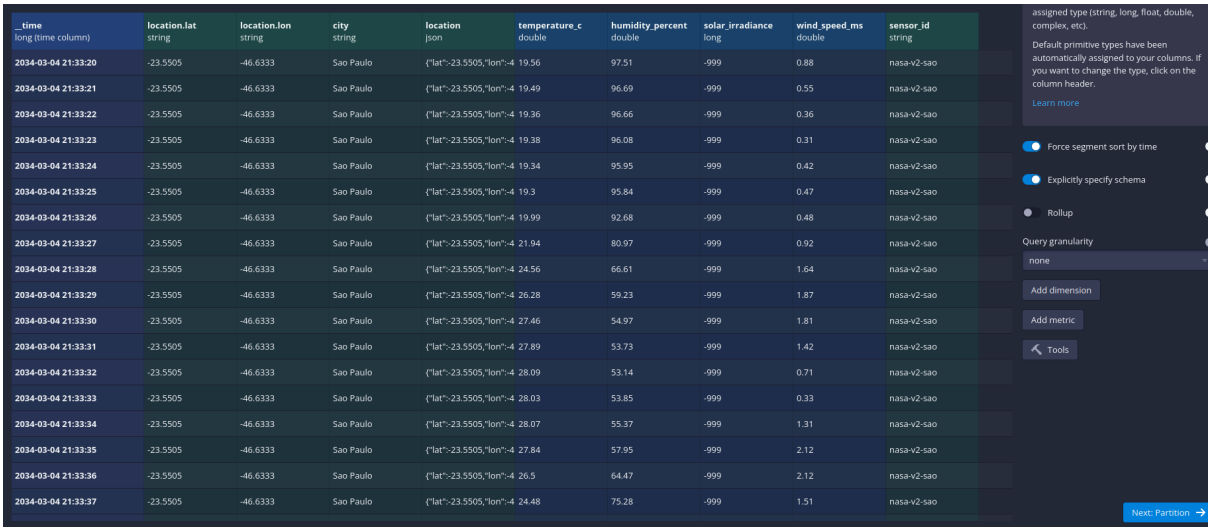


Figura 2 - Configuração do parser temporal para converter o formato numérico da NASA em Timestamp ISO.

Passo C: Definição do Schema e Rollup

Para garantir a integridade dos dados brutos para análises futuras, a funcionalidade de Rollup (pré-agregação) foi desativada. Isso garante que cada evento enviado pelo sensor seja armazenado individualmente.

- *Rollup:* OFF
- *Tipagem:* Métricas como temperature_c e solar_irradiance definidas explicitamente como Double/Float.



__time long (time column)	location.lat string	location.lon string	city string	location json	temperature_c double	humidity_percent double	solar_irradiance long	wind_speed_ms double	sensor_id string
2034-03-04 21:33:20	-23.5505	-46.6333	Sao Paulo	{"lat":-23.5505,"lon":-4	19.56	97.51	-999	0.88	nasa-v2-sao
2034-03-04 21:33:21	-23.5505	-46.6333	Sao Paulo	{"lat":-23.5505,"lon":-4	19.49	96.69	-999	0.55	nasa-v2-sao
2034-03-04 21:33:22	-23.5505	-46.6333	Sao Paulo	{"lat":-23.5505,"lon":-4	19.36	96.66	-999	0.36	nasa-v2-sao
2034-03-04 21:33:23	-23.5505	-46.6333	Sao Paulo	{"lat":-23.5505,"lon":-4	19.38	96.08	-999	0.31	nasa-v2-sao
2034-03-04 21:33:24	-23.5505	-46.6333	Sao Paulo	{"lat":-23.5505,"lon":-4	19.34	95.95	-999	0.42	nasa-v2-sao
2034-03-04 21:33:25	-23.5505	-46.6333	Sao Paulo	{"lat":-23.5505,"lon":-4	19.3	95.84	-999	0.47	nasa-v2-sao
2034-03-04 21:33:26	-23.5505	-46.6333	Sao Paulo	{"lat":-23.5505,"lon":-4	19.99	92.68	-999	0.48	nasa-v2-sao
2034-03-04 21:33:27	-23.5505	-46.6333	Sao Paulo	{"lat":-23.5505,"lon":-4	21.94	80.97	-999	0.92	nasa-v2-sao
2034-03-04 21:33:28	-23.5505	-46.6333	Sao Paulo	{"lat":-23.5505,"lon":-4	24.56	66.61	-999	1.64	nasa-v2-sao
2034-03-04 21:33:29	-23.5505	-46.6333	Sao Paulo	{"lat":-23.5505,"lon":-4	26.28	59.23	-999	1.87	nasa-v2-sao
2034-03-04 21:33:30	-23.5505	-46.6333	Sao Paulo	{"lat":-23.5505,"lon":-4	27.46	54.97	-999	1.81	nasa-v2-sao
2034-03-04 21:33:31	-23.5505	-46.6333	Sao Paulo	{"lat":-23.5505,"lon":-4	27.89	53.73	-999	1.42	nasa-v2-sao
2034-03-04 21:33:32	-23.5505	-46.6333	Sao Paulo	{"lat":-23.5505,"lon":-4	28.09	53.14	-999	0.71	nasa-v2-sao
2034-03-04 21:33:33	-23.5505	-46.6333	Sao Paulo	{"lat":-23.5505,"lon":-4	28.03	53.85	-999	0.33	nasa-v2-sao
2034-03-04 21:33:34	-23.5505	-46.6333	Sao Paulo	{"lat":-23.5505,"lon":-4	28.07	55.37	-999	1.31	nasa-v2-sao
2034-03-04 21:33:35	-23.5505	-46.6333	Sao Paulo	{"lat":-23.5505,"lon":-4	27.84	57.95	-999	2.12	nasa-v2-sao
2034-03-04 21:33:36	-23.5505	-46.6333	Sao Paulo	{"lat":-23.5505,"lon":-4	26.5	64.47	-999	2.12	nasa-v2-sao
2034-03-04 21:33:37	-23.5505	-46.6333	Sao Paulo	{"lat":-23.5505,"lon":-4	24.48	75.28	-999	1.51	nasa-v2-sao

Figura 3 - Definição do esquema de dados, garantindo a tipagem correta para operações matemáticas.

3.3. Aprofundamento Técnico: Arquitetura de Ingestão (Supervisor & Tasks)

Para garantir que a ingestão de dados via Streaming (Kafka) seja contínua e tolerante a falhas, o Apache Druid utiliza uma arquitetura mestre-escravo específica para indexação.

O Supervisor (O Orquestrador)

No Druid, a ingestão de streams não é feita por um processo único e monolítico. Ao submeter a *Ingestion Spec*, criamos um **Supervisor**.

- **Função:** O Supervisor é um processo leve que roda no *Overlord Service*. Ele não processa dados diretamente. Sua função é gerenciar o estado da leitura do Kafka.
- **Responsabilidades:**
  - **Gerenciamento de Offsets:** Ele monitora em qual mensagem (offset) a leitura parou para garantir que, se o sistema reiniciar, nenhum dado seja perdido ou duplicado (garantia *exactly-once* ou *at-least-once*).
  - **Auto-Healing:** Se um worker falhar, o Supervisor detecta e sobe outro imediatamente.
  - **Escalabilidade:** Se o tópico do Kafka tiver múltiplas partições, o Supervisor cria múltiplas Tasks para ler em paralelo.

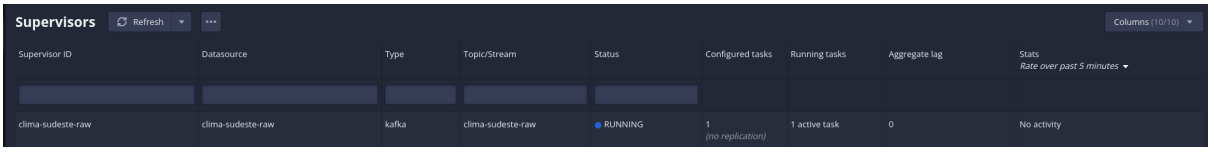


Figura 4 - Supervisors do Topico Kafka

As Tasks (Os Trabalhadores)

As **Tasks** (ou Tarefas de Indexação) são os processos efêmeros que executam o trabalho pesado. No caso do Kafka, elas são do tipo index_kafka.

- **Ciclo de Vida:** Uma Task é criada pelo Supervisor, roda por um período configurado (ex: 1 hora ou até atingir um tamanho de arquivo) e depois "morre", passando o bastão para uma nova Task.
- **O que ela faz:**
    1. Conecta no Broker do Kafka e consome as mensagens.
    2. Faz o *Parse* (lê o JSON).
    3. Aplica transformações e filtros.
    4. Constrói os **Segmentos** (arquivos colunares otimizados do Druid) em memória.
    5. Publica esses segmentos no *Deep Storage* (disco) para serem consultados pelos nós Históricos.

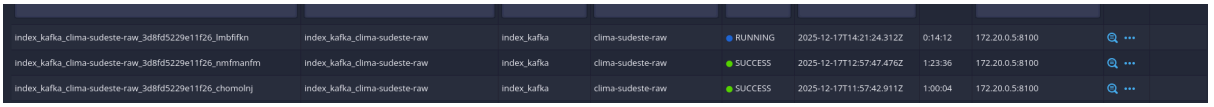


Figura 5 - Tasks index Kafka

---

4. Validação e Resultados

A integração foi validada através de consultas SQL nativas no console do Druid, confirmando a disponibilidade das quatro cidades monitoradas e das novas métricas de radiação solar.

Exemplo de Query de Validação:

SQL
```
SELECT "city", AVG("temperature_c"), MAX("solar_irradiance")
FROM "clima-sudeste-raw"
GROUP BY 1
```

A city	1.0 EXPR$1 AVG("temperature_c")	123 EXPR$2 MAX("solar_irradiance")
Belo Horizonte	23.64608333333334	-999
Itajuba	21.602458333333335	-999
Rio de Janeiro	26.752374999999997	-999
Sao Paulo	22.462625000000003	-999

Figura 4 - Consulta SQL comprovando a ingestão e a capacidade analítica do Druid sobre os dados do Kafka.

---

5. Próximos Passos: Benchmarking, Nuvem e Observabilidade

A próxima fase do projeto visa validar a arquitetura proposta através de testes de estresse e comparação quantitativa (Benchmarking).

5.1. Visualização e Controle (Data Viz)

Para interpretar os resultados do benchmark, a visualização de dados deixará de ser apenas sobre "Clima" e passará a incluir "Performance do Sistema".

- **Ferramenta:** Integração com **Apache Superset**.
- **Objetivo:** Construir um *Dashboard de Operações* que mostre visualmente:
  - **Latência de Ingestão:** O tempo decorrido entre a geração do dado no Python e sua disponibilidade para query no Druid (Lag).
  - **Taxa de Processamento:** Quantidade de linhas por segundo (Rows/s) ingeridas.
  - **Comparativo de Queries:** Gráficos de barra comparando o tempo de resposta de queries complexas (Agregações) entre Druid vs. PostgreSQL.

5.2. Migração para Nuvem (Cloud Robustness)

O ambiente local (Docker no localhost) não reflete as latências de rede e limitações de I/O de um cenário real.

- **Ação:** Realizar o *"Lift and Shift"* dos containers Docker atuais para uma instância de nuvem (ex: AWS EC2, Google Compute Engine ou Azure VM).

- **Teste de Robustez:**
    - Verificar o comportamento do sistema sob latência de rede real.
    - Testar a resiliência do *Supervisor* do Druid ao reiniciar o container do Kafka propositalmente na nuvem.
    - Avaliar o custo-benefício (Custo de infraestrutura vs. Volume de dados processados).

5.3. Telemetria "Ciclo Fechado" (Self-Monitoring)

Uma ideia seria utilizar o próprio Apache Druid para armazenar as métricas de seu funcionamento.

- **Conceito:** O script Python (Producer) passará a enviar, além dos dados climáticos, metadados sobre o envio (tamanho do payload, tempo de resposta da rede).
- **Métricas do Druid (Emitters):** Configurar o Druid para emitir suas próprias métricas internas (uso de CPU, uso de Heap Java, Garbage Collection) para um tópico Kafka separado (druid-metrics).
- **Ingestão Recursal:** Criar um novo *Datasource* no Druid chamado system_metrics que lê desse tópico.

5.4. Definição do Benchmark (Plano de Testes)

Será estabelecido um protocolo rigoroso para comparar o Apache Druid contra uma tecnologia controle (Baseline), provavelmete o PostgreSQL (já presente no docker-compose).

Métricas de Sucesso (KPIs) a serem coletadas:

Categoria	Métrica	Descrição
Ingestão	Throughput (Linhas/seg)	Quantos registros o banco aguenta escrever por segundo antes de travar ou criar lag.
Armazenamento	Taxa de Compressão	Comparar o tamanho em disco dos dados brutos (JSON) vs. Segmentos Druid vs. Tabelas SQL.

Consulta	Query Latency (ms)	Tempo de resposta para uma agregação pesada (ex: Média de temperatura de 10 milhões de linhas).
Concorrência	Queries por Segundo (QPS)	Como o tempo de resposta degrada se 50 usuários fizerem consultas ao mesmo tempo.